Contents

Chapter 1

Making a Map

Plotly is an open-source data visualization library for Python, R, and JavaScript. It allows for interactive plots, including geographical maps. In this brief example, we will learn how to create a simple annotated map using Plotly in Python.

To begin, you need to install Plotly. In Python, you can do this via pip:

```
pip install plotly
```

Once installed, you can create a map with annotations as follows:

```python
import plotly.graph_objects as go

fig = go.Figure(data=go.Scattergeo(
    lon = [-75.789],
    lat = [45.4215],
    text = ['Ottawa'],
    mode = 'text',
))

fig.update_layout(
    title_text = 'Annotated Map with Plotly',
    showlegend = False,
    geo = dict(
        scope = 'world',
        projection_type = 'azimuthal equal area',
        showland = True,
        landcolor = 'rgb(243, 243, 243)',
        countrycolor = 'rgb(204, 204, 204)',
    ),
)

fig.show()


```

This code creates a world map and places a text annotation at the geographic coordinates for Ottawa. Then, the plotly is opened in your browser using a localhost IP address. The ‘go.Scattergeo‘ function is used to define the geographical scatter plot (i.e., the annotation), while the ‘update_layout‘ function is used to define the appearance and the properties of the map itself.

In this example, you can replace the latitude, longitude, and text with the values corresponding to your desired location.

Chapter 2

Introduction to Discrete Probability

Before we begin talking about discrete probability, let's address the word *discrete*. It means "individually separate and distinct" as in "You can think of light as a continuous wave or as a blast of discrete particles."

When we are talking about probabilities, some problems deal with discrete quantities like "What is the probability that I will throw these three dice and the numbers that roll face up sum to 9?" There are also problems that deal with continuous properties like "What is the probability that the next bird to fly over my house will weigh between 97.2 and 98.1 grams?" In this module, we are going to focus on the probability problems that deal with discrete quantities.

Watch Khan Academy's Introduction to Probability at `https://youtu.be/uzkc-qNVoOk`.

Let's say that you have a cloth sack filled with 100 marbles; 99 are red and 1 is white. If you reach in without looking and pull out one marble, you will probably pull out a red one. We say that "There is a 1 in 100 chance that you would pull out a white marble." Or we can use percentages and say "There is a 1% chance that you will pull out a white marble." Or we can use decimals and say "There is a 0.01 probability that you will pull out a white marble." In probability, we often talk about the probability of certain events. "Pulling out a white marble" is an event, and we can give it a symbol like W. In equations, we use p to mean "the probability of". Thus, we can say "There is a 0.01 probability that you will pull out a white marble," which becomes the equation

$$p(W) = 0.01$$

2.1 The Probability of All Possibilities is 1.0

We know that you are either going to pull out a red marble or a white marble, so the probability of a white marble being pulled and the probability of a red marble being pulled must add up to 100%. Therefore, the odds of pulling out a red marble must be 99%, or 0.99. If we let the event "Pull out a red marble" be given by the symbol R, we can say:

$$p(R) = 1.0 - P(W) = 1.0 - 0.01 = 0.99$$

Now, let's say that you take a marble from the bag, then toss a coin. What is the probability that you will pull a white marble, then get heads on the coin? It is the product of the two probabilities: $0.01 \times 0.5 = 0.005$, so one-half of a one percent chance. Do the probabilities still sum to 1?

- White and Heads = $0.01 \times 0.5 = 0.005$
- White and Tails = $0.01 \times 0.5 = 0.005$
- Red and Heads = $0.99 \times 0.5 = 0.495$
- Red and Tails = $0.99 \times 0.5 = 0.495$

$0.005 + 0.005 + 0.495 + 0.495 = 1.000$ Yes, the probabilitites of all the possibilities still add to 1. This is only true because *the events do not effect each other*; the events are ***independent***.

2.2 Independence and Dependence

In the last section, you learned that the probability of two events ("Pulling a red marble from the bag" and "Getting tails in a coin toss") is the product of the probability of each event: $0.99 \times 0.5 = 0.495$.

This is true if the two events are *independent*; that is, the outcome of one doesn't change the probability of the other. The example we gave is independent: It doesn't matter what ball you pull from the bag, the outcome of the coin toss will always be 50-50.

What is an example of two events that are not independent? The probability that a person is a professional basketball player and the probability that someone wears a shoe that is size 13 or larger is *not* independent; rather it is dependent. After all, height is an advantage in basketball, and most tall people also have large feet. So, if you know someone is a basketball player, they likely wear large shoes.

2.3 Correlation does not Imply Causation

It is often said that "Correlation does not imply causation". What does this mean? Breaking it down:

- Correlation is a state in which two things increase or decrease together; two variables are *linearly* related
- Imply means suggests, as in *there is evidence for*
- Causation means there is a cause and effect relationship in place between the two variables.

So saying correlation does not imply causation simply means: just because two variables are linearly connected, it cannot be said one variable causes the other.

Here are some examples:

- Ice Cream Sales, Mosquito bites, and Sunburn cases all increase during the summertime. However, none of these are causally connected.
- Dinosaurs were not able to read books. However, this does not mean that the inability to read caused dinosaur extinction.
- There was a large increase in toilet paper sales and people working from home in the years 2020-2021. Does this mean that one causes the other? Not necessarily. In this case a third factor C, the pandemic, caused a large global quarantine, leading to increased toilet paper usage and the inability to work in an office.

Exercise 1 Rolling Dice

If you have three six-sided dice to roll, what is the probability that you will roll a five on all three dice?

Working Space

Answer on Page 29

Exercise 2 Flipping Coins

If you have five coins to flip, what is the probability that at least one coin will come up heads?

Working Space

Answer on Page 29

2.4 Why 7 is the most likely sum of two dice

If you roll two dice, the sum will be 2, 12, or any number in between. It is very tempting to assume that the likelihood of any of those numbers is the same. In fact, the probability of a 2 is $\frac{1}{36} \approx 3\%$ and the probability of a 7 is $\frac{1}{6} \approx 17\%$. A 7 is six times more likely than a 12! Why?

When you roll the first die, there are six possibilities with equal probability. When you roll the second die, there are six possibilities with equal probability. So, there are a total of 36 possible events with equal probabilities: 1 then 1, 1 then 2, 2 then 1, 1 then 3, 3 then 1, and so on. Only one of these (1 then 1) adds to 2. However, six of these sum to 7: 1 then 6, 6 then 1, 2 then 5, 5 then 2, 3 then 4, and 4 then 3. So, a 7 is six times more likely than a 2.

Here is the complete table:

Sum							Count	Probability
2	1,1						1	1/36
3	1,2	2,1					2	1/18
4	1,3	2,2	3,1				3	1/12
5	1,4	2,3	3,2	4,1			4	1/9
6	1,5	2,4	3,3	4,2	5,1		5	5/36
7	1,6	2,5	3,4	4,3	5,2	6,1	6	1/6
8		2,6	3,5	4,4	5,3	6,2	5	5/36
9			3,6	4,5	5,4	6,3	4	1/9
10				4,6	5,5	6,4	3	1/22
11					5,6	6,5	2	1/18
12						6,6	1	1/36

If you don't believe these numbers, you could roll a pair of dice hundreds of times and make a histogram. However, it would be a tedious and time-consuming task — just the sort of thing that we make computers do for us.

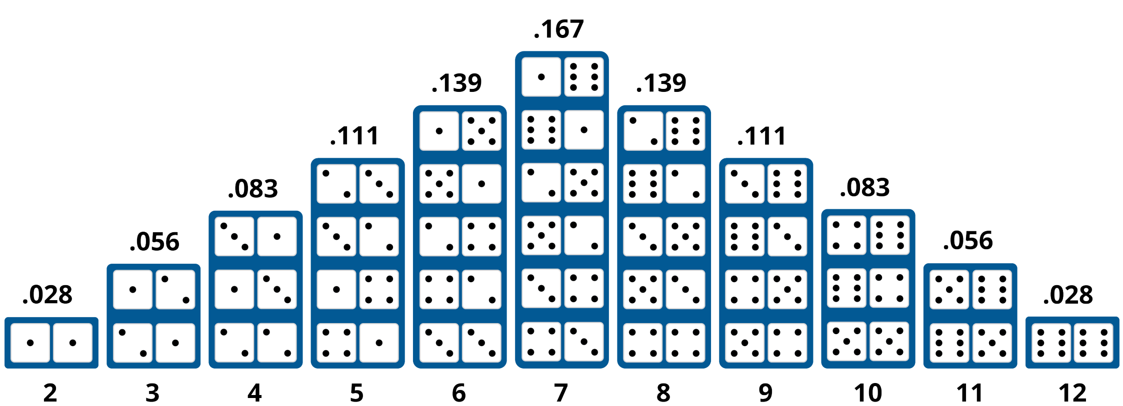

Figure 2.1: A histogram of probabilities of dice sums. This is referred to as a bell curve.

2.5 Random Numbers and Python

You are going to write a simulation of rolling dice in Python. To do this, you will need to generate a random sequence of numbers. The numbers will need to be in the range 1 to 6, and they will need to appear in the sequence with the same frequency. We say the sequence will follow *the uniform distribution*. In other words, the probability is uniformly distributed among the 6 possibilities.

Start python and try a few of the different ways to generate random numbers:

```
> python3
>>> import random
>>> random.random()  # Generates a random floating point number between 0 and 1
0.6840892758539989
>>> randrange(5)       # Generates an integer in the range 0 - 4
2
>>> x = ['Rock', 'Paper', 'Scissors']
>>> random.choice(x)   # Pick a random entry from the sequence
'Paper'
>>> x
['Rock', 'Paper', 'Scissors']
>>> random.shuffle(x)   # Shuffle the order of the sequence
>>> x
['Scissors', 'Paper', 'Rock']
>>> a = list(range(30))
>>> a
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
  16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 29]
>>> random.sample(a, 10) # Return 10 randomly chosen items from the sequence
[8, 7, 20, 9, 25, 13, 23, 11, 14, 16]
```

Clearly, Python has many ways to do things that look random. However, here's the hidden truth: they aren't really random. The computer that you are using can't generate random data. Instead, it uses tricks to create data that look random; we call this *pseudorandom* data. Good pseudorandom algorithms are very important for cryptography and data security.

What if you want real random data? Some companies are using the decay of radioactive materials to generate real random data. You can pay to download it. For our purposes, Python's pseudorandom numbers are quite sufficient.

If we generate two random numbers in the range 1 through 6 and add them together, we will have simulated rolling a pair of dice. Like this:

```
>>> a = random.randrange(6) + 1
>>> b = random.randrange(6) + 1
>>> a + b
```

```
8
```

First, let's write a program that just rolls the dice 100 times and shows the result. Make a file `dice.py`:

```python
import random

roll_count = 100

for i in range(roll_count):
    a = random.randrange(6) + 1
    b = random.randrange(6) + 1
    roll = a + b
    print(f"Toss {i}: {a} + {b} = {roll}")
```

When you run it, you should see something like:

```
> python3 dice.py
Toss 0: 6 + 6 = 12
Toss 1: 4 + 4 = 8
Toss 2: 4 + 2 = 6
Toss 3: 4 + 6 = 10
Toss 4: 4 + 4 = 8
...
Toss 98: 5 + 2 = 7
Toss 99: 5 + 2 = 7
```

Now we want to count occurrences of each possible outcome. Let's use an array of integers. We will start with an array of zeros. When we roll a 3, we will add 1 to item 3 in the array. (We can never roll a zero or a one, so those two entries will always be zero.)

```python
import random

roll_count = 100

# Make an array containing 13 zeros
counts = [0] * 13

for i in range(roll_count):
    a = random.randrange(6) + 1
    b = random.randrange(6) + 1
    roll = a + b
    print(f"Toss {i}: {a} + {b} = {roll}")

    # Increment the count for roll
    counts[roll] += 1
```

```

print(f"Counts: {counts")}
```

When you run this, at the end you will see a count for each possible outcome:

```
...
Toss 98: 3 + 2 = 5
Toss 99: 6 + 1 = 7
Counts: [0, 0, 2, 6, 16, 11, 13, 14, 11, 11, 6, 9, 1]
```

What was the count that we expected? For example, we expected to see a 2 about once every 36 rolls, right? It might be nice to compare our count to what we expected. Add a few more lines, and we are going to increase the number of rolls. You will probably want to delete the line that prints each roll separately:

```
import random

# Can't ever be 0 or 1
p = [0.0, 0.0, 1/36, 1/18, 1/12, 1/9, 5/36, 1/6, 5/36, 1/9, 1/12, 1/18, 1/36]
roll_count = 1000

# Make an array containing 13 zeros
counts = [0] * 13

for i in range(roll_count):
    a = random.randrange(6) + 1
    b = random.randrange(6) + 1
    roll = a + b

    # Increment the count for roll
    counts[roll] += 1

for i in range(2,13):
    print(f"{i appeared {counts[i]} times, expected {p[i] * roll_count:.1f}")}
```

Now you should see something like:

```
2 appeared 39 times, expected 27.8
3 appeared 55 times, expected 55.6
4 appeared 84 times, expected 83.3
5 appeared 110 times, expected 111.1
6 appeared 160 times, expected 138.9
7 appeared 176 times, expected 166.7
8 appeared 124 times, expected 138.9
9 appeared 93 times, expected 111.1
```

```
10 appeared 87 times, expected 83.3
11 appeared 49 times, expected 55.6
12 appeared 23 times, expected 27.8
```

Whenever you are dealing with random numbers, the outcome will seldom be *exactly* what you expected. In this case, however, you should see that your predictions are pretty close.

2.5.1 Making a bar graph

A bar graph is a nice way to look at quantities like this. Let's make a bar graph that shows the actual count and the expected count:

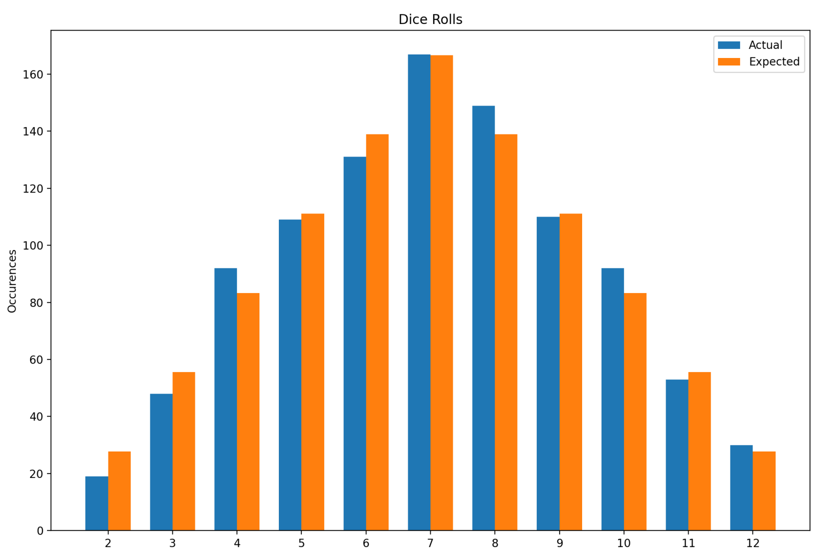

Figure 2.2: Dice roll histogram.

We need to describe the set of rectangles. To do this we will loop through each possible roll (2 - 12) and put data in four lists for each:

```
import random
import matplotlib.pyplot as plt

# Can't ever be 0 or 1
p = [0.0, 0.0, 1/36, 1/18, 1/12, 1/9, 5/36, 1/6, 5/36, 1/9, 1/12, 1/18, 1/36]
roll_count = 1000

```

```
# Make an array containing 13 zeros
counts = [0] * 13

for i in range(roll_count):
    a = random.randrange(6) + 1
    b = random.randrange(6) + 1
    roll = a + b

    # Increment the count for roll
    counts[roll] += 1

# Gather data for bar chart
bar_width = 0.35
expected = []
actual_starts = []
expected_starts = []
labels = []
actual = []
for i in range(2,13):
    expected.append(p[i] * roll_count)
    actual.append(counts[i])
    actual_starts.append(i - bar_width/2)
    expected_starts.append(i + bar_width/2)
    labels.append(i)

fig, ax = plt.subplots()

# Create the bars
ax.bar(actual_starts, actual, bar_width, label='Actual')
ax.bar(expected_starts, expected, bar_width, label='Expected')
ax.set_xticks(labels)

# Provide labels
ax.set_ylabel('Occurences')
ax.set_title('Dice Rolls')
ax.legend()
plt.show()
```

Chapter 3

Beginning Combinatorics

Discrete probability problems often include some counting. For example, we figured out that there were 36 different ways the two dice could land, but all of them summed to some number 2 through 12. How many different ways could three eight-sided dice come up? We would need to count them, right? As the numbers get bigger, we will need some tricks so that we don't need to write them all down and count them one-by-one.

The branch of mathematics that focuses on tricks for counting is called *combinatorics*.

How can we be sure that there were 36 different configurations for the two 6-sided dice? The first die could have come up as any one of six numbers. For each of those, the second could have come up with any one of six numbers. Thus, the number of possibilities is $6 \times 6 = 36$.

How many different configurations for three 8-sided dice? $8 \times 8 \times 8 = 8^3 = 512$.

What about seven dice, each with 20 sides? There would be $20^7 = 1,280,000,000$ configurations. See, aren't you glad we don't need to write them all down?

Now, let's say that six people (Anne, Brock, Carl, Dev, Edgar, and Fred) are going to run a race. You have to make a plaque that says who won first place, who won second place, and who won third. If you want to get all the possible plaques created beforehand, and just pull the right one out as soon as the race ends, how many plaques would you need to get engraved?

Since you're making these plaques before the race even happens, you need one plaque for every possible outcome. So think of it as filling in three blanks: first place, second place, and third place. Any of the 6 people could fill the first blank. Whoever fills that blank can't also fill the second blank — a person can't come in both first *and* second — so only 5 people are left to fill the second blank. Likewise, only 4 people remain to fill the third blank, since two people have already been assigned to the first two spots. Multiplying these together, $6 \times 5 \times 4 = 120$, tells you exactly how many different first-second-third outcomes are possible — and thus how many plaques you'd need engraved.

What if the plaque includes all 6 places? Using the same reasoning, you'd fill in six blanks instead of three: $6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$ plaques engraved. We use this process often enough that we gave it a name. We say "I need 6 factorial plaques engraved." When we write a *factorial*, we use an exclamation point:

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$

We use the word "permutation" to mean a particular ordering. This rule says n items can be ordered in $n!$ ways. Thus, mathematicians actually say "If you have a list of n items then we can generate $n!$ different permutations of those items". We will discuss this further in the next chapter.

In Python, there is a `factorial` function in the math library:

```
> python3
>>> import math
>>> math.factorial(6)
720
```

Handy, right? Now you don't need to write a loop to calculate factorials.

Remember when we only wanted the first three names on the plaque? We can solve that problem using factorials:

$$6 \times 5 \times 4 = \frac{6 \times 5 \times 4 \times 3 \times 2 \times 1}{3 \times 2 \times 1} = \frac{6!}{3!}$$

This formulation makes it easy to figure out on any calculator with a "!" button.

The rule on this is to fill m positions from n items. It can be done this many ways:

Permutations

For permutations where order is a factor, the formula for ${}^nP_m$ (n objects pick m) is

$$\frac{n!}{(n-m)!}$$

3.1 Choose

Let's say that there are 12 kids in a classroom and you need a team of four to clean the top of the desks. How many different possible teams are there? You know that if you were giving out four different positions (Like the race gave out 1st, 2nd, and 3rd), the answer would be $12 \times 11 \times 10 \times 9$ or $12!/(12-4)!$.

However, once we pick the 4 people, we don't care what order they are in, right? In this problem, the team "Anne, Brad, Carl, and Don" is the same as the team "Carl, Don, Brad, and Anne".

Thus, the quantity $12!/(12-4)!$ is many times too large, because it counts each permutation separately. To get the right number, we just divide this by the number of possible permutations for a group of four people: $4!$

That gets us our answer: How many different teams of four can be chosen from 12 people?

$$\frac{12!}{(12-4)!4!} = 495$$

Generally, this formula, referred to as the *Binomial coefficient*. In combinatorics, we use this quantity a lot, so we have given it a name: *choose*

Combinations

For combinations, where order doesn't matter, the formula for ${}^{n}C_{k}$ (n objects choose k) is

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

We have also given it a notation. "12 choose 4" is written like this:

$$\binom{12}{4}$$

Python has the `math.comb` function:

```
> python3
>>> import math
>>> comb(12, 4)
495
```

We briefly saw this in the common polynomial products chapter. The binomial theorem says that the coefficients of the terms in the expansion of $(a+b)^n$ are given by the binomial coefficients $\binom{n}{k}$.

$$(a+b)^4 = 1a^4+4a^3b+6a^2b^2+4ab^3+1b^4 = \binom{4}{0}a^4+\binom{4}{1}a^3b+\binom{4}{2}a^2b^2+\binom{4}{3}ab^3+\binom{4}{4}b^4$$

3.2 Fractional Binomial Theorem

An issue arises when we have fractional exponents for this theorem. For example, what is $\sqrt{1+5x} = (1+5x)^{1/2}$? We can use the binomial theorem to expand this as follows:

$$(1+5x)^{1/2} = \binom{1/2}{0}1 + \binom{1/2}{1}5x + \binom{1/2}{2}(5x)^2 + \binom{1/2}{3}(5x)^3 + \ldots$$

Pascal's triangle only works when the exponent n is a non-negative integer. Here, the exponent is $\frac{1}{2}$, not a whole number. There's no "row $\frac{1}{2}$" of Pascal's triangle, so we need to extend the pattern.

This extension is called the ***general binomial theorem***. To make sense of $\binom{1/2}{k}$, we need a definition of the binomial coefficient that doesn't rely on factorials, since $(1/2)!$ doesn't make sense. Recall that

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n(n-1)(n-2)\cdots(n-k-1)}{k!}$$

Notice that the right-hand side only ever multiplies and divides — it never actually requires n to be a whole number! This gives us a definition that works for *any* real number n, including fractions and negative numbers:

Generalized Binomial Coefficient

For any real number n and non-negative integer k,

$$\binom{n}{k} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k!}$$

When n is *not* a non-negative integer, like $n = \frac{1}{2}$, none of the factors $n, n-1, n-2, \ldots$ is ever zero, so the expansion never terminates. Instead of a finite row of Pascal's triangle, we get an infinite series.

General Binomial Theorem

Recall that for $|x| < 1$, the geometric series gives

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \cdots$$

Since $\frac{1}{1-x} = (1-x)^{-1}$, this looks like a binomial expansion where the exponent -1 is not a positive integer. Using the generalized binomial coefficient,

$$\binom{n}{k} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k!}$$

For any real number n and $|x| < 1$,

$$(1+x)^n = \sum_{k=0}^{\infty} \binom{n}{k} x^k = 1 + nx + \frac{n(n-1)}{2!}x^2 + \frac{n(n-1)(n-2)}{3!}x^3 + \cdots$$

this is called the *binomial series*, and it generalizes the binomial theorem to arbitrary real exponents. Setting $n = -1$ recovers the geometric series above.

The condition $|x| < 1$ matters here for convergence. When n was a positive integer, the expansion was just a polynomial, and polynomials are perfectly well-behaved for every value of x. But now that we have an infinite series, we need the terms to actually get smaller and smaller so the sum settles down to a finite value. If $|x| \geq 1$, the terms don't shrink, and the series doesn't converge to anything useful.

Let's compute the coefficients for $\binom{1/2}{k}$

$$\binom{1/2}{0} = 1$$
$$\binom{1/2}{1} = \frac{1}{2}$$
$$\binom{1/2}{2} = \frac{\frac{1}{2}\left(\frac{1}{2}-1\right)}{2!} = \frac{\frac{1}{2}\cdot\left(-\frac{1}{2}\right)}{2} = -\frac{1}{8}$$
$$\binom{1/2}{3} = \frac{\frac{1}{2}\left(-\frac{1}{2}\right)\left(-\frac{3}{2}\right)}{3!} = \frac{3/8}{6} = \frac{1}{16}$$

So,

$$(1+x)^{1/2} = 1 + \frac{1}{2}x - \frac{1}{8}x^2 + \frac{1}{16}x^3 - \cdots$$

Now we can go back to our original problem, $\sqrt{1+5x} = (1+5x)^{1/2}$. This is just $(1+x)^{1/2}$ with x replaced by $5x$, so we substitute $5x$ everywhere:

$$\begin{aligned}\sqrt{1+5x} &= 1 + \frac{1}{2}(5x) - \frac{1}{8}(5x)^2 + \frac{1}{16}(5x)^3 - \cdots \\ &= 1 + \frac{5}{2}x - \frac{25}{8}x^2 + \frac{125}{16}x^3 - \cdots\end{aligned}$$

This expansion is only valid for $|5x| < 1$, i.e. $|x| < \frac{1}{5}$.

Exercise 3 Fractional Binomial Expansion

Working Space

1. Find the first four terms in the expansion of $(1+x)^{-1}$, in ascending powers of x. For what values of x is this expansion valid?
2. Find the first three terms in the expansion of $\sqrt{1-2x}$, in ascending powers of x.

Answer on Page 29

3.3 Applications in Probability

Suppose you flip a fair coin 5 times. What's the probability that you get exactly 3 heads?

Note that there are only two options: Heads (H) or Tails (T). Thus, there are $2^5 = 32$ possible sequences of flips. Each sequence is equally likely, so the probability of any particular sequence is $\frac{1}{32}$. Only a certain number of sequences will have exactly 3 heads.

One way to get 3 heads is HHHTT. There's also HTHHT, THHHT, and many other orderings also give you 3 heads. Before we can find the probability, we need to know *how many* different sequences of 5 coin flips contain exactly 3 heads. As we have discussed, we can use the n-choose-k formula. There are

$$\binom{5}{3} = \frac{5!}{3!2!} = 10$$

such sequences. This is said ”5 choose 3”, or, more explicitly “get 3 successes in 5 trials”.

Each individual sequence, like HHHTT, has probability $\left(\frac{1}{2}\right)^3\left(\frac{1}{2}\right)^2$ of occurring, since each flip is independent. Since there are 10 equally likely sequences that give exactly 3 heads, we add up 10 copies of that probability:

$$P(\text{exactly 3 heads}) = \binom{5}{3}\left(\frac{1}{2}\right)^3\left(\frac{1}{2}\right)^2 = 10 \cdot \frac{1}{32} = \frac{10}{32} = \frac{5}{16}$$

This pattern of counting the arrangements with $\binom{n}{k}$, then multiplying by the probability of one arrangement works whenever you have repeated *independent* trials with only two possible outcomes each time (success or failure).

Exercise 4 Free Throws

A free throw shooter has a past record of making 60% of her free throws. Over the next 6 free throws, what is the probability that she makes exactly 4 of them?

Working Space

Answer on Page 29

This probability situation has been rightfully called the *binomial distribution*. This is a brief introduction to the binomial distribution, and we will discuss it in more detail in a later chapter.

Chapter 4

Permutations and Sorting

In the previous chapter, we talked about permutations. If you have a list of four letters, like $[a, b, c, d]$, you can rearrange them in 4! ways:

a,b,c,d	a,b,d,c	a, d, b, c	a, d, c, b	a, c, b, d	a, c, d, b
b,a,c,d	b,a,d,c	b, d, a, c	b, d, c, a	b, c, a, d	b, c, d, a
c,b,a,d	c,b,d,a	c, d, b, a	c, d, a, b	c, a, b, d	c, a, d, b
d,b,c,a	d,b,a,c	d, a, b, c	d, a, c, b	d, c, b, a	d, c, a, b

You can make Python generate all the permutations for you:

```
from itertools import permutations
all_permutations = permutations(('a', 'b', 'c', 'd'))
for p in all_permutations:
    print(p)
```

4.1 Notation

How do we define or write down a single permutation? You could say something like "Swap the first and second items and swap the third and fourth items." However, that gets pretty difficult to read, so we usually write a permutation as two lines: the first line is before the permutation and the second line is after. Like this:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 4 & 3 \end{pmatrix}$$

We can also assign permutations to variables. For example, if we wanted the variable A to represent "swapping the first and second item", we would write this:

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 3 & 4 \end{pmatrix}$$

And if we wanted B to represent "swapping the third and fourth item", we would write:

$$B = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 4 & 3 \end{pmatrix}$$

Now, we can *compose* permutations together. For example, we might say:

$$B \circ A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 4 & 3 \end{pmatrix}$$

In other words, if we have the list $[a, b, c, d]$ and we apply permutation A, followed by permutation B, we get $[b, a, d, c]$.

Important: Note that permutations are applied from right to left. $B \circ A$ means "Applying A and then B." Why does this matter? Permutations are not necessarily commutative. That is, if you have two permutations S and T, $S \circ T$ is not always the same as $T \circ S$.

Also, note that "don't change anything" is a permutation. We call it *the identity permutation*. If you have four items, the identity permutation would be written:

$$I = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \end{pmatrix}$$

(We use a capital "I" for the identity.)

4.1.1 Challenge

Find an example of two permutations S and T, such that $S \circ T$ does not equal $T \circ S$.

4.2 Sorting in Python

One of the common forms of permutation in software is sorting. Sorting is putting data in a particular order. For example, in Python, if you had a list of numbers, you can sort it in ascending order like this:

```
my_grades = [92, 87, 76, 99, 91, 93]
grades_worst_to_best = sorted(my_grades)
```

Do you want to sort backwards?

```
my_grades = [92, 87, 76, 99, 91, 93]
grades_best_to_worst = sorted(my_grades, reverse=True)
```

Note that `sorted` makes a new list with the correct order. If you want to sort the array in place, you can use the `sort` method:

```
my_grades = [92, 87, 76, 99, 91, 93]
my_grades.sort(reverse=True)
```

4.3 Inverses

Think for a second about this permutation:

$$S = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 4 & 2 & 1 \end{pmatrix}$$

You could say this permutation shuffles a list. What is its inverse? That is, what is the permutation that unshuffles the items back to where they were originally?

$$S^{-1} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 4 & 3 & 1 & 2 \end{pmatrix}$$

In other words, the original moved an item in the first spot to the third spot; the inverse must move whatever was in the third spot back to the first spot.

(Notation note: In multiplication, $b \times b^{-1} = 1$, so we use "to the negative one" to indicate inverses in lots of places.)

Mechanically, how do you find the inverse? Flip the rows, then sort the columns using the top number:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 4 & 2 & 1 \end{pmatrix} \text{ flip } \rightarrow \begin{pmatrix} 3 & 4 & 2 & 1 \\ 1 & 2 & 3 & 4 \end{pmatrix} \text{ sort } \rightarrow \begin{pmatrix} 1 & 2 & 3 & 4 \\ 4 & 3 & 1 & 2 \end{pmatrix}$$

Let's say you have two permutations A and B. Permuting by B then A would look like this:

$$C = A \circ B$$

If you know A^{-1} and B^{-1}, what is C^{-1}? You would undo-A, then undo-B, so

$$C^{-1} = B^{-1} \circ A^{-1}$$

4.4 Cycles

Here is a permutation:

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 4 & 5 & 1 & 3 \end{pmatrix}$$

When this is applied, whatever is at 1 gets moved to 2, 2 gets moved to 4, and 4 gets moved to 1. That is a *cycle*: $1 \to 2 \to 4$, then it goes back to 1. It involves three locations, so we say it is a *3-cycle*.

There is another cycle in this permutation: $3 \to 5$, then it goes back to 3.

Because these cycles share no members, we say the cycles are *disjoint*.

Every permutation can be broken down into a collection of disjoint cycles.

$$T = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 4 & 5 & 1 & 3 \end{pmatrix} = (1 \to 2 \to 4)(3 \to 5)$$

The first handy thing about this notation is that it makes it easy for us to describe the inverse. We just run the cycles backward:

$$T^{-1} = (4 \to 2 \to 1)(5 \to 3)$$

Starting with the list $[a, b, c, d, e]$, let's repeatedly apply the permutation T

Initial	a, b, c, d, e
T applied	d, a, e, b, c
$T \circ T$ applied	b, d, c, a, e
$T \circ T \circ T$ applied	a, b, e, d, c
$T \circ T \circ T \circ T$ applied	d, a, c, b, e
$T \circ T \circ T \circ T \circ T$ applied	b, d, e, a, c
$T \circ T \circ T \circ T \circ T \circ T$ applied	a, b, c, d, e

This permutation results in six combinations, then it loops back on itself. The number of combinations is the least common multiple of all the cycles. In this case, there is a 3-cycle and a 2-cycle. The least common multiple of 2 and 3 is 6.

4.5 Further Learning

Check out this leetcode problem which combines permutations and programming! https://leetcode.com/problems/permutations/description/

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 7)

probability of all 5's $= \frac{1}{6} \times \frac{1}{6} \times \frac{1}{6} = \left(\frac{1}{6}\right)^3 = \frac{1}{216} \approx 0.0046$

Answer to Exercise 1 (on page 7)

probability of at least one heads = 1.0 - probability of all tails $= 1.0 - \left(\frac{1}{2}\right)^5 = 1.0 - \frac{1}{32} = \frac{31}{32} = \approx 0.97$

Answer to Exercise 3 (on page 20)

1. $(1+x)^{-1} = 1 - x + x^2 - x^3 + \cdots$, valid for $|x| < 1$

2. $\sqrt{1-2x} = 1 - x - \frac{1}{2}x^2 - \cdots$

Answer to Exercise 4 (on page 21)

$$P(\text{exactly 4 makes}) = \binom{6}{4}(0.6)^4(0.4)^2 = 0.31104$$

Index